Project Report

Sound Recognition and Visualization

Electrical, Computer, and Biomedical Engineering Department
Abu Dhabi University

Amro Kamal Jalad 1092505

Supervised By: Dr. Yazan Dweiri

جامعة أبوظبي
ABU DHABI UNIVERSITY

Contents

1 Introduction **5**
- 1.1 Motivation . . . 5
- 1.2 Problem Statement . . . 6
- 1.3 Literature Review . . . 6

2 Design **7**
- 2.1 Requirements Constraints, and Considerations . . . 7
- 2.2 Design Process . . . 8
  - 2.2.1 GUI Layout and Recording Module . . . 9
  - 2.2.2 Signal Analysis and Filtering Module . . . 10
- 2.3 System Overview . . . 11
- 2.4 Component Design . . . 13

3 GUI Overview and Mathematical Foundations **15**
- 3.1 GUI Overview . . . 15
- 3.2 Mathematical Foundations and Course Connections . . . 16
  - 3.2.1 Sampling and Discrete-Time Signals . . . 16
  - 3.2.2 Fourier Series and Harmonic Structure . . . 17
  - 3.2.3 Discrete Fourier Transform . . . 17
  - 3.2.4 LTI Systems and Convolution . . . 18
- 3.3 Key Code Walkthrough . . . 19

4 Experimental Testing and Results **21**
- 4.1 Testing Plan and Acceptance Criteria . . . 21
- 4.2 Results . . . 22
  - 4.2.1 Test 1 and Test 2: Normal and Silent Recordings . . . 22
  - 4.2.2 Test 3 and Test 4: Noise Filtering . . . 24
  - 4.2.3 Test 5: Load Raw Data File . . . 26
- 4.3 Analysis and Interpretation of Data . . . 26

5 Conclusion **27**
- 5.1 Summary . . . 27
- 5.2 Future Improvements and Takeaways . . . 28
- 5.3 Lessons Learned . . . 28
- 5.4 Individual Work Reflection . . . 29
- 5.5 Impact Statement . . . 29

A MATLAB Source Code **30**
A.1 soundRecognitionGUI.m 30
A.2 analyzeSignal.m 44
A.3 filterNoise.m 49

List of Figures

1 The block diagram of the GUI for Sound Recognition and Visualization system. 12
2 Completed GUI; left are the control panels, right are the six analysis plots. 15
3 All six analysis plots are shown in this example of normal speech recording for Test 1. 23
4 Test 2: near-silent recording, to check that the system is able to process low energy input properly. 24
5 Test 3: deliberately noisy recording, unfiltered. 25
6 In the following, the same recording from Test 3 is analyzed again, but this time with the bonus noise filter set at sharpness 15. 25

List of Tables

1 Chart of course topics and system components. 16
2 The influence of the bonus noise filter on a deliberately noisy recording. 24

Abstract

This report presents the design and implementation of a Sound Recognition and Visualization system, a MATLAB Graphical User Interface (GUI) that records a short audio message and analyzes it in both the time and frequency domains. The system displays the recorded waveform, its Fourier Transform, its Power Spectrum, its Power Spectral Density, a histogram of its amplitude, and a spectrogram of how its frequency content changes over time, alongside the signal's energy, dominant frequency, and core statistical features (mean, standard deviation, variance). A bonus noise-filtering stage, implemented as a windowed-sinc bandpass FIR filter with an adjustable sharpness control, was also built and tested. Five categories of test recordings were used to validate the system: a normal speech recording, a near-silent recording, a deliberately noisy recording, a sharpness sweep of the noise filter on that noisy recording, and a verification that a previously saved raw data file reproduces identical results when reloaded. Results confirmed expected signal-processing behavior throughout: the Fourier Transform correctly concentrated energy in the speech-relevant frequency range, the silent recording produced near-zero energy and variance, and the noise filter measurably reduced signal energy as its sharpness increased, while also revealing a known limitation of frequency-domain filtering when noise and speech share overlapping frequency content. The project demonstrates a complete, working, end-to-end application of the Fourier Transform and statistical signal analysis concepts covered in the Signals and Systems course.

1 Introduction

In everyday engineering practice, ranging from speech recognition systems to telephone systems, voice assistants to hearing aids, audio signals are among the most common types of signals that are met. All of these functions are essential to every phone call, voice command, hearing aid adjustment: sampling an ongoing sound wave, analyzing its frequency content, and deriving meaningful parameters from it. Knowing how to record, visualize and manipulate a meaningful feature of an audio signal bridges the gap between the theoretical concepts of Signals and Systems, such as the Fourier Transform, and a real, tangible application. The same process of recording, transforming and interpreting a signal is used in much more sophisticated systems – noise-cancelling headphones, automatic speech recognition, to name just two – and a small, compact version of this process is a valuable system to build and understand from scratch. The concepts in this project are applied in the construction of a MATLAB based Graphical User Interface (GUI) which records a short spoken message and performs analysis of it in the time and frequency domains, following work on a foundational Signals and Systems textbook [1], a classic approach to spectral estimation [2] and official MATLAB documentation [3].

1.1 Motivation

Most of the applications using voice, such as smart assistants or biomedical monitoring devices, use the same general workflow: record an audio signal, analyse the underlying frequency spectrum, and then extract features to define it. As a practical engineering tool, the working version of this pipeline will give students direct experience of the uses of theoretical Fourier analysis. Now, it's one thing to get the Fourier Transform of an idealized, textbook example of a signal and the answer is known beforehand, and everything works out perfectly, it's another thing entirely to apply the same transform to a real recording of one's own voice, a recording that invariably contains background noise, imperfect timing, and a spectrum that doesn't always behave the way a clean example would suggest. Such a user-friendly visualization tool also provides educational value; a user can now instantly visualize the relationship between a spoken sentence and a spectrum and a power distribution and a set of statistical descriptors, without considering the Fourier Transform as some abstract equation. The visualization of the mapping is immediate, on

call, as a sentence the user just spoke; it is good to build intuition about what actually is the Fourier Transform than a repetition of this derivation on another set of numbers.

1.2 Problem Statement

Raw audio recordings (from a microphone) contain no understandable information on their own except for the amplitude over time. The waveform itself tells little about the frequencies present, the amount of energy in the signal, or how the amplitude is distributed across the frequencies; all of that information is in the recording but it is hidden in the waveform. Finding the frequency content, energy, and statistical structure of a recording is currently achieved using either specialized commercial tools for audio editing, or manual scripting, none of which have a common interface, and most of which are designed to be professional tools rather than tools designed to clearly expose the quantities of the signal processing that a Signals and Systems student would like to see. This project fills that void by creating one MATLAB GUI that will enable the user who has no signal-processing background to record a message and immediately get both the time domain plot and the frequency domain plot of that message, and also the statistical summary of the message, without having to write a single line of code.

1.3 Literature Review

Like with any treatment of the Fourier Transform, this project's foundation is the mathematical fact that any finite-energy time domain signal can be broken down into its frequency components and that the breakdown can be efficiently computed in the sample domain (discrete time) with what is known as the Fast Fourier Transform (FFT), a foundation established by Oppenheim and Willsky's classic treatment of the subject [1]. To overcome this, a practical challenge of the raw FFT is its sensitivity to noise; Welch's periodogram-averaging technique [2] addresses this by averaging spectral estimates over multiple segments of a signal, and a number of spectrogram implementations adopt a related short-time FFT approach, following the short-time spectral analysis framework developed by Allen in 1977 [4], where the estimate of the spectrum of a signal is averaged over segments of the signal, thereby reducing the noise sensitivity of the FFT. Windowing a finite window before each FFT for the spectrogram and filter design is done using

Hann and Hamming windows, following Harris' analysis of windowing functions in 1978 [5] which indicates that windowing has the effect of significantly decreasing spectral leakage when compared to no windowing. The built-in functions used for recording and analysing audio are outlined in the project documentation [3], which is provided by MathWorks. The fact that the signal being analyzed in this project is human speech and not some arbitrary signal makes Rabiner and Schafer's foundational text on speech processing [6] directly relevant; the text suggests that voiced speech is quasi-periodic, and that the spectrum of speech has a harmonic structure which justifies the choice of frequency ranges for the noise filter and the interpretation as a Fourier Series of the harmonic peaks observed during testing (Section 3.2.2). The system constructed in this project is deliberately more limited than a full-featured commercial tool like Audacity, however: it does not include all the features found in a professional audio editor, but only those signal processing quantities relevant to a Signals and Systems course, leaving the more complex filtering features out.

2 Design

The implementation of the system was done completely within MATLAB, with the use of the GUI framework `uifigure`/`uigridlayout` of MATLAB [3] and using the audio recording functions of MATLAB and calculations for signal processing for which the code was coded. No third-party toolboxes were necessary: all calculations (FFT-based spectrum, power spectrum, power spectral density, energy, spectrogram, and noise filter) were done directly with base MATLAB functions like `fft` or `conv` and can be executed on any standard MATLAB installation.

2.1 Requirements Constraints, and Considerations

The design requirements were directly taken from the project description and course rubric:

- The system should be programmed in MATLAB.
- The GUI should enable an end-user who has no prior signal processing knowledge to record audio messages.

- The GUI should contain the recorded signal in the time domain.
- The GUI should display the Fourier Transform, power spectrum and power spectrum density of the recorded signal.
- GUI should give the energy of the signal, highest frequency component and statistical properties of the signal (Mean, Standard Deviation, Variance).
- The GUI needs to show a histogram of the amplitude of the signal collected.
- As a bonus requirement it should filter noise out of the recording, if desired, before visualization and analysis.
- The GUI should enable users to load an existing audio file from their devices and analyze it using the same pipeline that would be used if it were an audio recording. This will allow for independent verification of the raw data files.

Concerning the bigger picture, the system wasn't dependent on any special hardware beyond a standard microphone, and thus doesn't add any extra electronic waste or specialized equipment expenses to the total. It was found that the system captures audio from the user, which is why a privacy concern was raised – the audio is the user's, it is stored locally in `.wav` format and is controlled by the user, not transmitted to another location by the application.

2.2 Design Process

The design process started with a mapping all the bullet points of the project description directly to a MATLAB function or GUI component, covering the required deliverables before any polishing work was completed. The original version employed MATLAB's Signal Processing Toolbox functions (periodogram, hamming, designfilt) in calculating the PSD and the bonus noise filter; however, this version did not work on machines that do not have the Signal Processing Toolbox installed, so the design was changed to calculate the PSD directly from the magnitude of the FFT and to manually construct the noise filter. There were five iterations of the noise filter design before one was found that reduced the noise without significantly affecting the voice: a simple moving-average filter, which smoothed noise but also

muffled speech since it could not distinguish noise from voice; a telephone-bandwidth filter (300–3400 Hz), which cleaned the signal but made it sound thin and unnatural; a wider bandpass filter (80–8000 Hz) which was too close to the full voice spectrum and did not produce audible noise reduction; a spectral-subtraction approach that measured and removed the specific noise profile of a recording but had little effect when the recording environment was already quiet; and finally the bandpass filter (150–5000 Hz) which was found to produce a clearly audible noise reduction while keeping the voice as natural as possible, regardless of the recording environment. A slider for the sharpness of the filter was added to allow the precision to be optimized and "tuned" by modifying it and observing the result in real time. A second change was the addition to the GUI of an extra plot area for the Power Spectrum, added by reorganizing the plot area into a 3x2 matrix of six plots; this was not there in an early version of the GUI, but was computed in the GUI and just wasn't visible. The "Load Audio File" option was added in a later revision, which means that an existing raw `.wav` file could be analyzed, as well as a live recording. First, a file drag-and-drop from the desktop to the window was explored, but a reliable efficient implementation of this in base MATLAB was not found, so a standard MATLAB file-picker dialog (uigetfile) was used instead, as this can be used to achieve the same practical effect: select an existing audio file and analyse it, but without relying on non-standard or version-sensitive workarounds.

2.2.1 GUI Layout and Recording Module

The initial implementation was limited to the recording and layout part of the GUI: a resizable window with a record-button, a play-button, a reset-button, a "Load Audio File"-button, a duration-field, a noise-filter control-panel, a save/export control-panel and 6 plotting-axes.

1. First, the recording callback records an audio segment of a specified duration with the MATLAB built-in recorder:

```
recorder = audiorecorder(Fs,16,1);
recordblocking(recorder,recTime);
x = getaudiodata(recorder);
```

2. The recorded signal is saved in a common variable that is available to

all other callbacks in the GUI, therefore pressing Play, Save or toggling the noise filter always affects the latest recording.

3. In step three, the layout was implemented using the MATLAB `uigridlayout` function, meaning that the interface will resize properly, and each control group (Recording, Noise Filter, Save and Export, Statistics) will be in its own named panel.

4. When a user clicks on the Load Audio File button, it will call the uigetfile routine to open a standard file picker and load the selected file using the routine audioread, with the subsequent analysis using the same sample rate as the loaded file; a file sampled at a different rate will be correctly handled:

```
[loadedSig, loadedFs] = audioread(fullfile(path,file));
x = loadedSig;
Fs = loadedFs;
updateDisplay(x);
```

2.2.2 Signal Analysis and Filtering Module

For the second stage the analysis functions were programmed and linked to the recording module, thus enabling the automatic re-analysis of the recorded data (or the noise filter) each time a new recording is made.

1. The recorded samples are used directly for Step one and the Fourier Transform and Power Spectrum are calculated direct:

```
X = fft(sig);
mag = abs(X);
f = (0:N-1)*(Fs/N);
half = 1:floor(N/2);
powerSpec = mag.^2 / N;
```

2. The maximum frequency component, energy, and statistical features are derived from the same FFT result and the raw samples, the Power Spectral Density is calculated by dividing the Power Spectrum by the sampling frequency, and a spectrogram is constructed by performing the FFT on a stack of short overlapping frames.

3. The signal is then convolved with the bonus noise filter (a windowed-sinc band pass FIR filter, 150-5000 Hz), prior to re-analysis of the signal with the same functions as the original recording.

2.3 System Overview

Figure 1 shows the overall architecture of the system. The user interacts with the GUI either by recording a new clip or by loading an existing audio file, both of which activate the same signal-processing pipeline. The recorded or loaded signal is sent to the analysis module where it calculates the time, frequency and statistical results and pushes them to the six plots and statistics panel of the GUI. When the noise-filter option is on, the signal is filtered through the noise filtering module before being input into the analysis module to be displayed on the screen.

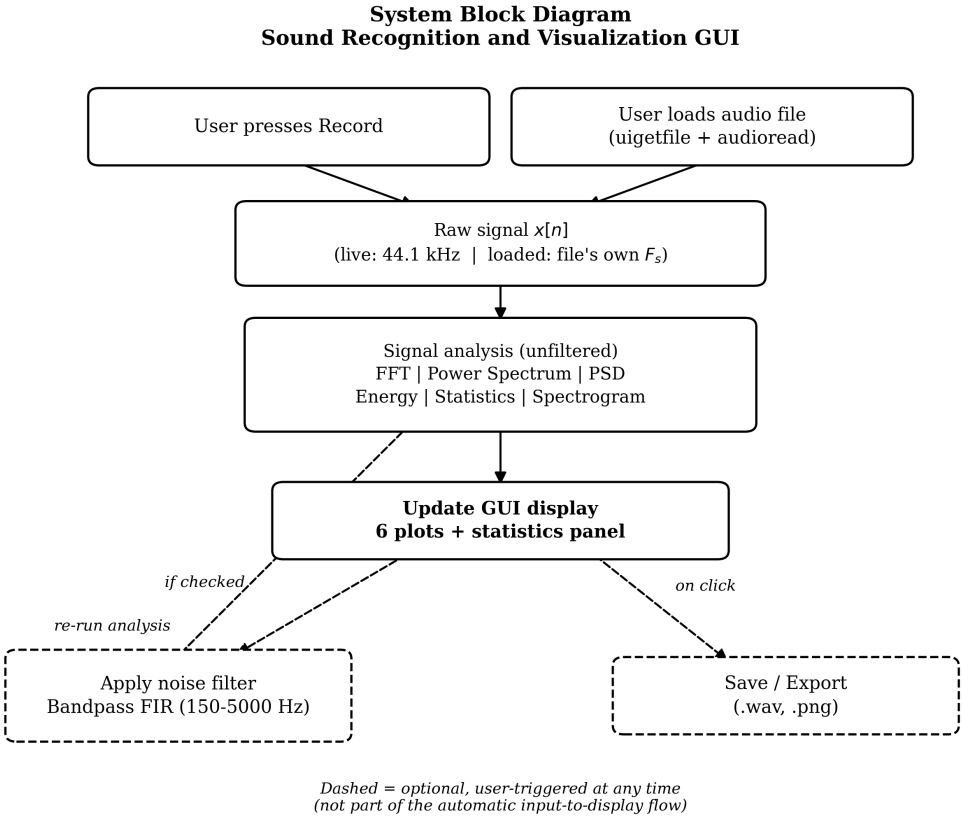


Figure 1: The block diagram of the GUI for Sound Recognition and Visualization system.

2.4 Component Design

The Fourier Transform component is used to transform the signal recorded from the time domain to the frequency domain using MATLAB's built-in FFT algorithm. Only the first half of the resulting spectrum is plotted, as the FFT of a real valued signal is symmetrical; the second half carries no additional information about the recording and would only take up screen space and obscure the meaningful portion of the spectrum.

This is called the Power Spectrum and is basically the distribution of the power of the signal over frequency, obtained by squaring the magnitude of the FFT and dividing by the number of samples. The Fourier Transform plot displays the amplitude strength of each frequency and the Power Spectrum is the same information, but in terms of power, which is the more physically meaningful quantity when comparing the amount of the total energy of a signal contained in a single frequency band. The Power Spectral Density also divides this quantity by the sampling frequency so that it has units of power per unit frequency; this allows two recordings of differing lengths to be compared on a level playing field.

The Maximum Frequency component is found by locating the single strongest peak in the FFT magnitude, but with everything below 50 Hz ignored first. This threshold was added after an early version of the code kept returning nonsense values, such as a "maximum frequency" of only a few Hertz, because low-frequency rumble and drift picked up by the microphone was consistently louder than the actual voice content. Since real human speech does not meaningfully start until around 85 Hz, cutting off everything below 50 Hz removes that mic noise without risking the loss of any genuine speech content, and the reported Maximum Frequency then correctly reflects the strongest true frequency component of the recorded voice.

```
minFreq = 50;
magSearch = mag;
magSearch(f < minFreq) = 0; % zero out anything below 50Hz first
[~, idx] = max(magSearch);
maxFreq = f(idx);
```

The Energy is calculated directly from the time-domain samples, as per Parseval's theorem in [1], and is the sum of the squared amplitudes. This is a helpful cross check of the frequency domain calculations as the same number can be obtained by summing up the Power Spectrum at all frequencies, which

is what Parseval's Theorem guarantees. The statistical parameters (mean, standard deviation, variance) are obtained from built-in functions in MATLAB: `mean`, `std` and `var` while the histogram is created using `histogram`. These, combined, provide a purely statistical fingerprint of the amplitude distribution of the recording that is independent of the time and frequency.

The spectrogram is a slight modification of the standard short-time spectral analysis [4] and uses a Hann window [5] and an FFT to each 512-sample frame of the signal. Rather than displaying the raw FFT magnitude directly, the values are first converted to a decibel scale using $20\log_{10}(|X|)$ before being drawn as a heat map in the time versus frequency plane. This step matters in practice: on a linear scale, the loudest moments of the recording would completely wash out the quieter detail, making most of the spectrogram look empty; the logarithmic scale compresses this range so that quiet and loud energy are both visible together, which is what actually lets the user visualize individual syllables of speech as time-frequency energy bands.

```
imagesc(ax_spec, r.specT, r.specF, 20*log10(r.specData + eps));
```

The `+ eps` is a small safeguard: without it, any completely silent frame would compute $\log_{10}(0)$, which is undefined, and MATLAB would return $-\infty$ instead of a plottable number. The spectrogram displays the frequencies present and when they were present, whereas with a single FFT of the entire recording you only know which frequencies were present at some point or other in the recording.

The bonus noise filter is a windowed-sinc bandpass FIR filter, designed to capture the energy of human speech in the range of 150–5000 Hz (the bulk of human speech energy [6]), manually designed from the sinc function and a Hamming window (no toolbox functions), and then applied using convolution. The passband and the rejected band are separated by an adjustable slider which corresponds to the filter order, therefore it is possible to pass up a filtered band to a rejected band with a gentle slope or an aggressive one; no code changes are required.

3 GUI Overview and Mathematical Foundations

3.1 GUI Overview

The final GUI looks like this (Figure 2). All controls are gathered together in four panels on the left side and the six live plots are shown on the right side, each panel labelled for the group of controls. If the user wants to record a clip for a fixed duration, pressing Record will record a clip; if the user instead wants to analyze an existing audio file rather than record a new one, they can press the Load Audio File button to select an existing file (in `.wav` format, `.mp3` format, `.m4a` format, or `.flac` format) from the user's device, and run it through the same analysis pipeline. The Duration field, the noise-filter checkbox and sharpness slider, and the Save/Export buttons allow the user to adjust all the necessary inputs without having to write any code.

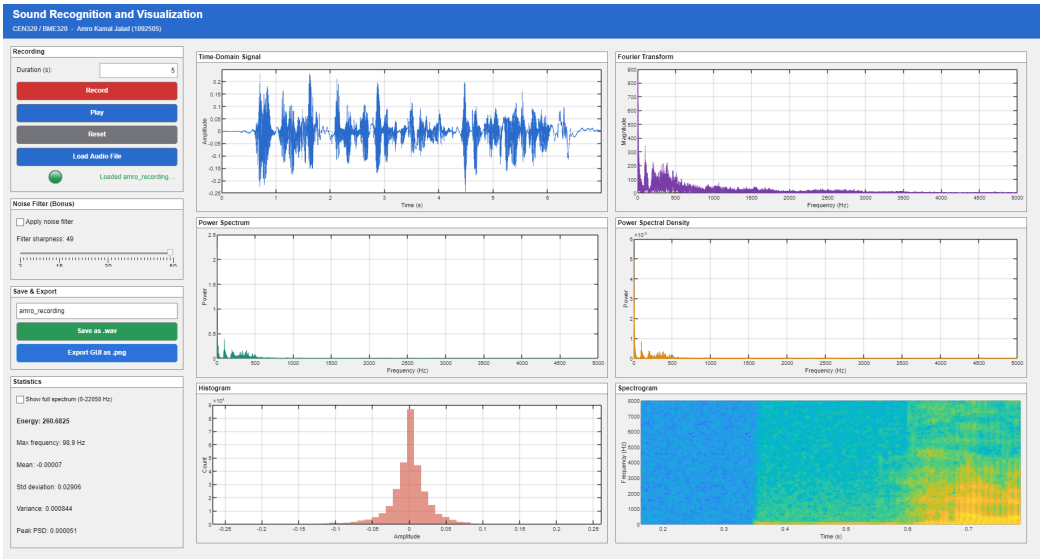


Figure 2: Completed GUI; left are the control panels, right are the six analysis plots.

3.2 Mathematical Foundations and Course Connections

Table 1 maps each component of the system to a topic listed in the course syllabus.

Table 1: Chart of course topics and system components.

Course Topic	**Applied In This Project**
Signals and Systems (energy, power)	Energy calculation, statistical features
LTI Systems	The bonus noise filter: completely specified by the impulse response and convolution
Fourier Series	The Fourier Series and the structure of harmonic speech are clearly apparent, as a regular series of peaks in the PSD output.
Fourier Transform	FFT, Power Spectrum, Power Spectral Density, spectrogram
Laplace Transform	Continuous-time filter design theory, the frequency response of the discrete FIR filter is modeled after that of the continuous-time filter.
Discrete-Time Signals and Systems	Sampling ($F_s = 44100$ Hz), discrete convolution, the DFT

3.2.1 Sampling and Discrete-Time Signals

The microphone signal is sampled at $F_s = 44100$ Hz, thus the signal is captured in discrete time as $x[n]$:

$$x[n] = x_c(nT_s), \qquad T_s = \frac{1}{F_s},$$

Here, $x_c(t)$ is the voice signal to begin with. Because the energy in human speech is low for frequencies above approximately 8 kHz [6] (well below the Nyquist frequency $F_s/2 = 22050$ Hz), the sampling rate satisfies the Nyquist criterion [1] and there is no aliasing. This limit is not just theoretical in this project: the GUI includes a "Show full spectrum" toggle that lets the user

zoom the Fourier Transform, Power Spectrum, and PSD plots out from the default 0–5000 Hz speech-focused view all the way to $F_s/2 = 22050$ Hz, the exact Nyquist frequency, so the ceiling imposed by the sampling rate can be seen directly rather than only calculated on paper.

```
if chk_fullspec.Value
    freqZoom = Fs/2; % exactly the Nyquist frequency
else
    freqZoom = 5000;
end
```

3.2.2 Fourier Series and Harmonic Structure

If the speech is voiced, e.g., vowel sounds, the vibration of the vocal cords is quasi-periodic and short portions of the recorded speech will have fundamental period T_0 that is approximately periodic [6]. According to Fourier Series theory, any periodic signal can be expressed as a sum of fundamental frequency and its integer multiple harmonics:

$$x(t) = \sum_{k=-\infty}^{\infty} c_k \, e^{j2\pi k f_0 t}, \qquad f_0 = \frac{1}{T_0}.$$

This could be seen directly during testing by zooming in on the Power-Spectral-Density plot using the interactive zoom feature of the GUI (zooming to approximately 0-1400 Hz): The resulting plot showed a comb-like pattern of evenly spaced peaks instead of a single smooth curve – each peak corresponds to a harmonic kf_0 of the fundamental frequency of the voice signal, a direct experimental verification of the Fourier series representation of a periodic signal.

3.2.3 Discrete Fourier Transform

The Fourier Transform, Power Spectrum, and Power Spectral Density are all derived from the Discrete Fourier Transform (DFT), computed efficiently in MATLAB using the Fast Fourier Transform (FFT) algorithm:

$$X[k] = \sum_{n=0}^{N-1} x[n] \, e^{-j2\pi kn/N}, \qquad k = 0, 1, \ldots, N-1.$$

The magnitude $|X[k]|$ is graphed directly as the Fourier Transform. The Power Spectral Density further divides by F_s and the Power Spectrum is $|X[k]|^2/N$. The Energy calculation is a simple application of Parseval's theorem, which tells us that a signal's energy is equal in the time domain and frequency domain.

$$E = \sum_{n=0}^{N-1} |x[n]|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |X[k]|^2.$$

The DFT is also applied to short frames of 512 samples, instead of the entire signal, which also causes a fundamental time-frequency resolution tradeoff [7]. A smaller frame has a finer resolution in time and a coarser resolution in frequency (the spacing between the DFT bins), with the tradeoff being that individual syllables will show up as vertical bands, rather than as well-defined pieces of the spectrum. A good example of this is that a 512-sample frame for $F_s = 44100$ Hz has a frequency spacing of $\Delta f \approx 86$ Hz, and a time resolution of just a few milliseconds. A longer frame would focus the frequency axis, but would merge together fast changes in time. In this project the 512-sample choice is selected for the purpose of time resolution; this is because the ability to distinguish between the four separate phrases in the test sentence in time was judged to be more useful for this application than resolving very closely spaced frequency components.

3.2.4 LTI Systems and Convolution

The bonus noise filter is an LTI system, and is completely defined by its impulse response $h[n]$. It's output is generated by Discrete Convolution with the filter $h[n]$:

$$y[n] = (x * h)[n] = \sum_k x[n-k]\, h[k].$$

The impulse response $h[n]$ of the filter is constructed as a difference between two ideal lowpass impulse responses (both sinc function, inverse Fourier Transform of an ideal rectangular frequency response), followed by a Hamming window to prevent ringing. This directly depends on the Fourier Transform pair of a rectangular frequency response with a sinc shaped impulse response, and brings the discrete time design of the filter back to the Fourier Transform properties, and in a conceptual manner, to the theory of designing continuous time filters using the Laplace Transform.

Two additional LTI system properties from the course are directly relevant here: First, the $h[n]$ as initially computed is symmetric about $n = 0$ and thus is non-causal (it relies on "future" samples) – the implementation corrects for this by simply delaying the output, eliminating the same number of leading samples that the filter's half-length adds with `conv`. Second, $h[n]$ is a finite-length filter (called a Finite Impulse Response, or FIR, filter), which means that the sum of the absolute value of each $h[n]$ for all n over the finite support is always finite, and therefore, is guaranteed to satisfy the standard BIBO stability condition for LTI systems.

The length of $h[n]$ – its filter order – is set directly by the sharpness slider in the GUI, using the mapping:

```
order = round(v)*6 + 1; % always odd
```

where `v` is the slider position, restricted to the range 3 to 50. This gives an odd-length filter order ranging from 19 samples at the lowest sharpness up to 301 samples at the highest. At the default sharpness of 15, the order is 91, which at $F_s = 44100$ Hz corresponds to an impulse response spanning roughly 2.1 milliseconds – concretely, this is how long the filter's own "memory" of past input samples actually lasts. A longer $h[n]$ gives the filter more samples to work with when separating the passband from the stopband, which is precisely why increasing the slider produces a sharper cutoff: the filter order and the impulse response length are the same underlying quantity, expressed once as an abstract LTI system property and once as a real, adjustable number in the running application.

3.3 Key Code Walkthrough

The most salient lines in each file, along with the mathematics that they implement.

From the DFT and Energy: `analyzeSignal.m` program:

```
X = fft(sig);            % X[k], the Discrete Fourier Transform
mag = abs(X);             % |X[k]|, plotted as the Fourier Transform
powerSpec = mag.^2 / N;  % |X[k]|^2 / N, the Power Spectrum
energy = sum(sig.^2);    % sum |x[n]|^2, energy via Parseval's
    theorem
```

These four lines perform the same DFT, Power Spectrum, and Energy equa-

tions above practically as written mathematically – `fft` computes $X[k]$, and the rest of the lines are the same as the mathematical equations for the Power Spectrum and Parseval's theorem.

In the filterNoise.m file, convolution and the impulse response:

```
hLow2 = 2*fc2/Fs * sincManual(2*fc2/Fs*n); % lowpass impulse
    response, cutoff fc2
hLow1 = 2*fc1/Fs * sincManual(2*fc1/Fs*n); % lowpass impulse
    response, cutoff fc1
hBand = hLow2 - hLow1;                     % bandpass impulse
    response h[n]
yFull = conv(sig, hBand);                  % y[n] = (x * h)[n]
pad = (order-1)/2;
y = yFull(pad+1 : pad+length(sig));        % trims the output back
    to the original length
```

The first three lines generate $h[n]$ from two lowpass sinc responses as discussed above and the fourth line is the convolution equation applied directly through the MATLAB function `conv` that is the defining operation of any LTI system. The last two lines are what actually implement the causality fix described in Section 3.2.4: because `conv` produces an output longer than the input by `pad` samples on each side, trimming off exactly `pad` samples from the front is the same as delaying the filter's originally non-causal, symmetric impulse response into a causal one, while also cutting the result back down to the original signal length.

One subtlety hidden inside `sincManual` is worth pointing out: mathematically, $\text{sinc}(x) = \sin(\pi x)/(\pi x)$ is a $0/0$ indeterminate form at exactly $x = 0$, since both the numerator and denominator vanish there. This is a removable singularity, not a real discontinuity – the actual limit as $x \to 0$ is 1, which follows directly from the small-angle behaviour of $\sin(\pi x)$. Evaluating the formula literally at $x = 0$ in MATLAB would instead return `NaN`, since floating-point division treats $0/0$ as undefined regardless of the true limit, so `sincManual` explicitly checks for $x = 0$ and substitutes the correct limiting value of 1 rather than letting the division fail. This single point matters because $n = 0$ always falls exactly in the middle of the symmetric range used to build $h[n]$, so without this fix the very centre sample of every impulse response the filter builds would silently be `NaN`.

4 Experimental Testing and Results

4.1 Testing Plan and Acceptance Criteria

To validate the functionality of this system, five types of tests were designed based upon test sentence "Hello, My Name is Amro, and my degree program is Computer Engineering. Welcome to ECBE at ADU."

- **Test 1 – Normal Speech Recording.** Description: Check that the system can correctly record, plot and analyze a normal spoken sentence. Steps: record the test sentence at normal volume. Expected Results: There will be a clear waveform, a spectrum focused in the speech range, and finite, non-zero statistics. Acceptance Criteria: No errors in rendering all 6 plots and statistics are physically plausible for speech. Test Result: **Pass**.

- **Test 2 – Silent Recording.** Description: verifies that the system doesn't lose the signal if the sound is at a low level. Steps: don't speak, just record. Expected Results: almost flat waveform, energy and variance near zero. Acceptance Criteria: absence of run-time errors while using the GUI for analysis. Test Result: **Pass**.

- **Test 3 – Noisy Recording, Filtered vs. Unfiltered.** Description: verifies whether the bonus noise filter makes a measurable difference in the analysis. Steps: record with purposeful background noise, then compare the unfiltered to the filtered. Expected Results: The filtered signal has less energy and a smooth spectrum. Acceptance Criteria: reduction in energy as seen and measured after filtering. Test Result: **Pass**.

- **Test 4 – Filter Sharpness Sweep.** Description: A test to see that the adjustable sharpness control of the filter acts as expected. Steps: apply the filter to the Test 3 recording for sharpness values of 5, 15, 35 and 50, keeping the statistics recorded each step. Expected Results: At each setting of sharpness, the filtered energy should be less, but not necessarily strictly less, than the unfiltered energy, depending on the location of the particular background noise in the passband of the filter. Acceptance Criteria: All the filtered results have lower energy compared to the unfiltered baseline, and any non-monotonic behavior can be explained by the filter theory. Test Result: **Pass**.

- **Test 5 – Load Raw Data File.** Description: ensures that the results of the analysis of a .wav file loaded from the Load Audio File button match those of the same file obtained in the original live recording so that the raw data can be checked independently. Steps: save a live recording to file `amro_recording.wav` then reset the GUI and then load it again from the Load Audio File button. Expected Results: All of the statistics (Energy, Max Frequency, Mean, Standard Deviation, Variance, Peak PSD) should match the original recording exactly. Acceptance criteria: All statistics are the same as displayed statistically with no difference. Test Result: **Pass**.

4.2 Results

4.2.1 Test 1 and Test 2: Normal and Silent Recordings

This section contains two types of test recordings – one normal and one silent. The first test, "normal", is a spoken recording that is normal. The waveform in Figure 3 indicates four distinct bursts of amplitude (the four phrases of the test sentence) occurring at regular intervals with short breaks between. Both the Fourier Transform and the Power Spectrum have energy below about 1000 Hz which is expected from human speech, and the Energy has been recorded as 260.6825, the Maximum Frequency at 98.9 Hz, and Standard Deviation 0.02906.

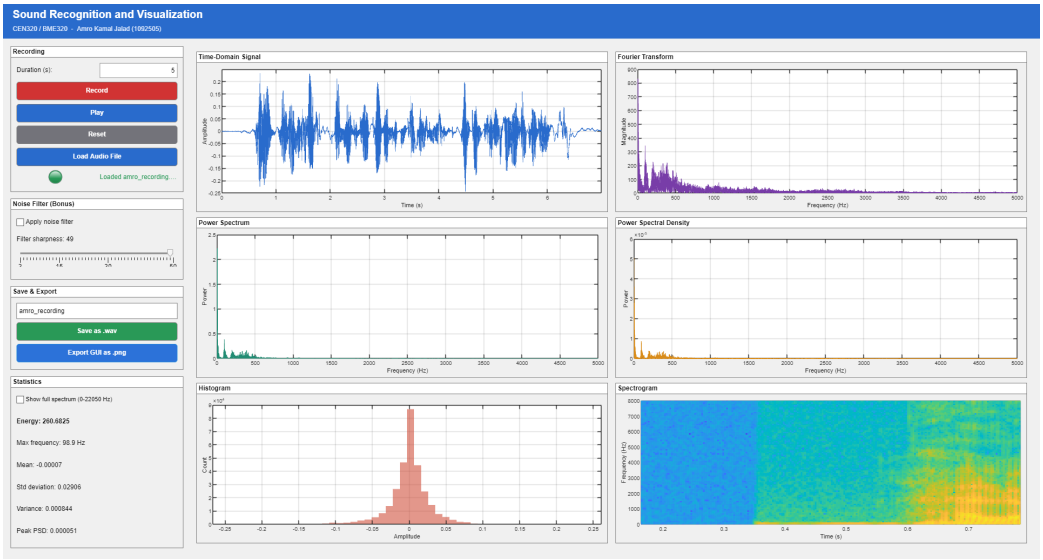


Figure 3: All six analysis plots are shown in this example of normal speech recording for Test 1.

The result of Test 2, with no speech present, is shown in Figure 4. The resulting statistics – Energy of 13.6158, Standard Deviation of 0.00786, and Variance of 0.000062 – are still approximately 19 times less than the normal speech recording, indicating that the system is able to distinguish speech from non-speech even though some ambient background noise was acoustically captured during the recording process and not absolute silence.

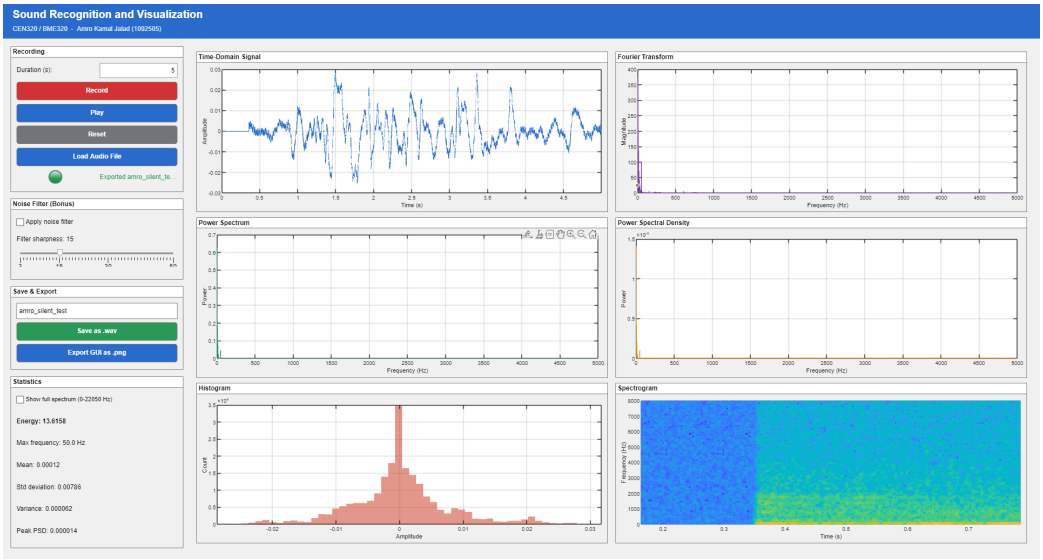


Figure 4: Test 2: near-silent recording, to check that the system is able to process low energy input properly.

4.2.2 Test 3 and Test 4: Noise Filtering

In Test 3 and Test 4 the sound is being filtered in different ways. Table 2 displays the statistics taken over this noisy test recording, as well as the unfiltered and four filter sharpness recordings.

Table 2: The influence of the bonus noise filter on a deliberately noisy recording.

Condition	Energy	Max Freq. (Hz)	Std. Dev.
Unfiltered	262.8837	390.0	0.02918
Filtered, sharpness 5	212.4435	390.0	0.02623
Filtered, sharpness 15	184.3611	532.3	0.02444
Filtered, sharpness 35	201.5900	390.0	0.02555
Filtered, sharpness 50	210.3501	390.0	0.02610

The unfiltered noisy recording is presented in Fig. 5 and the same with the filter applied at sharpness 15 is shown in Fig. 6.

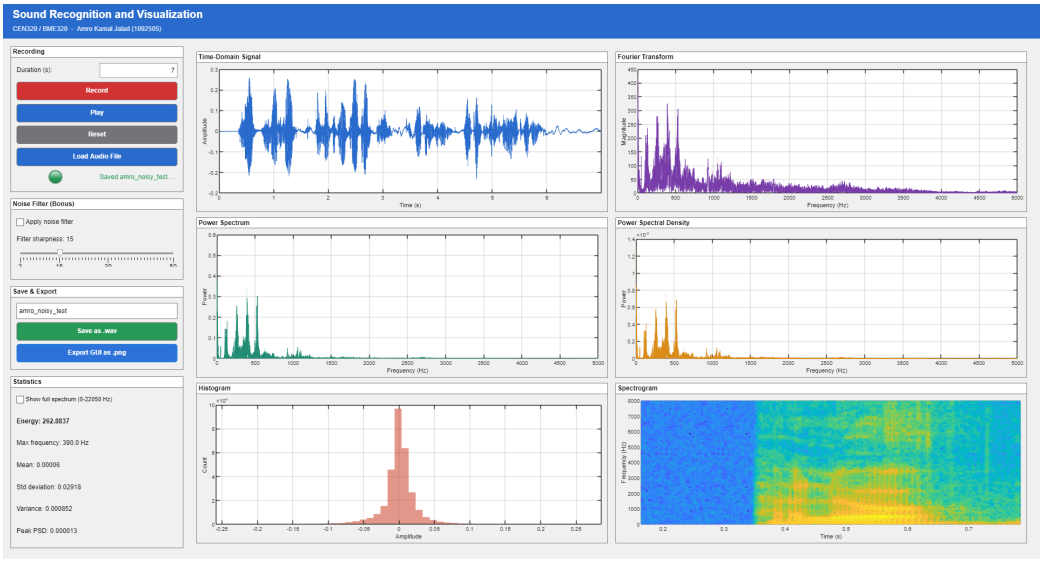


Figure 5: Test 3: deliberately noisy recording, unfiltered.

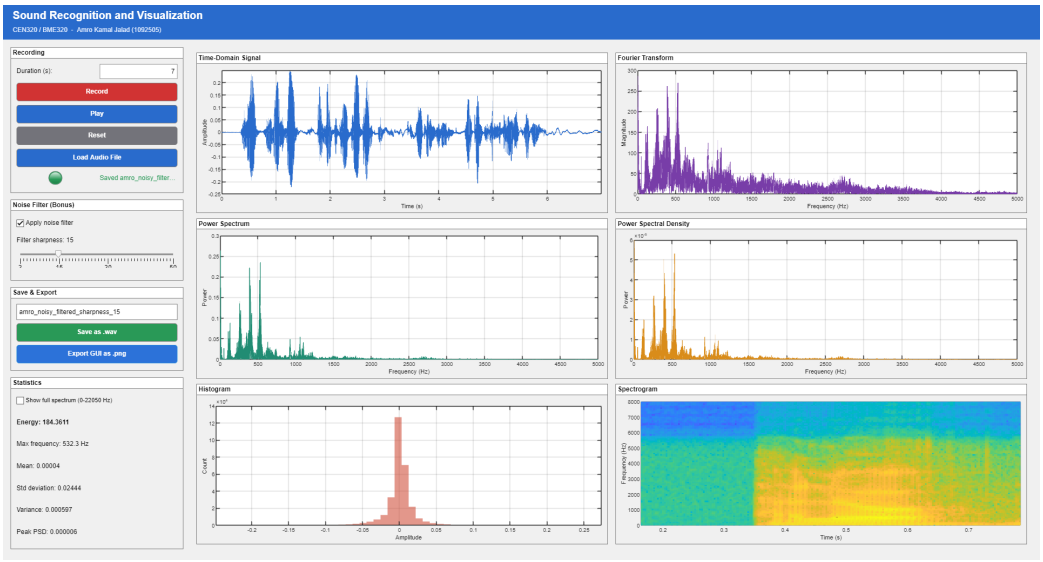


Figure 6: In the following, the same recording from Test 3 is analyzed again, but this time with the bonus noise filter set at sharpness 15.

4.2.3 Test 5: Load Raw Data File

The status label shown in Figure 3 (which reads "Loaded amro_recording...." rather than "Recording complete" to confirm) indicates that this figure was not a new recording, but rather a loading of the recording file amro_recording.wav back into the GUI via the Load Audio File button. The statistics produced are, digit by digit, the same as the ones found in the original live recording of the same file, that is Energy = 260.6825, Maximum Frequency = 98.9 Hz, Mean = −0.00007, Standard Deviation = 0.02906, Variance = 0.000844 and Peak PSD = 0.000051. This is to confirm that the analysis pipeline is the same for both live recording and loaded file and that the submitted raw `.wav` data can be reloaded and tested independently to give the same results as reported in this document.

4.3 Analysis and Interpretation of Data

The trend is clearly and consistently visible throughout the results in table 2 and one additional point of detail that might be explained. First, the Energy of each of the filtered versions is less than that of the unfiltered version (262.88 unfiltered vs 212.44, 184.36, 201.59 and 210.35 at sharpness 5, 15, 35 and 50 respectively), so the filter is a reliable way to remove signal energy from all sharpnesses.

Second, Energy is not monotonically decreasing with Sharpness – it is at its minimum at SHARPNESS = 15, but increases slightly at 35 and 50, but not further. This is not a mistake, but is an indication that this is a fixed band pass filter (150–5000 Hz) not a retuned filter tuned to the noise. The amount of noise eliminated from the input is a function of the frequency of the background noise used in this experiment and the transition band of a windowed-sinc FIR filter is not strictly linear as a function of its order. The same effect is seen in the Maximum Frequency reading, where it remained at 390.0 Hz for sharpness settings 5, 35 and 50, but rose to 532.3 Hz when it occurred at sharpness 15 – the setting at which the filter's transition occurred in a position that eliminated the 390-hz component sufficiently so that the highest remaining component was subsequently a 532.3-hz tone. This is the same as filter theory in that the sharpness of the transition between the passband and stopband can be controlled with the order of the filter (see [1] and [7]), but that is not a linearly related curve, and this experiment illustrates a real-world curve, not an idealized textbook curve.

Another discovery was that the bandpass filter did not completely remove the background noise heard in Test 3, which could be heard as well as read from the plots. The limitation is an expected one: The deliberate noise injected during the tests was in the same frequency band as the speech, so a fixed bandpass filter was not able to separate the noise from the speech, because it can only remove energy *outside* its passband. An adaptive technique like spectral subtraction, which was also tested and implemented in development, can remove noise, but has little effect in cases of recording a noise source in a quiet, unchanging environment, because it can only subtract noise that is in the measured noise profile. The characteristic of a bandpass filter – that it will have a useful effect in virtually any environment, whereas an adaptive filter will only work when there is true background noise to remove – was a very practical Signals and Systems knowledge that was discovered through testing, and not something that the design of such a filter could have predicted.

5 Conclusion

5.1 Summary

Each of the objectives and design constraints described in the project have been met in this work and a GUI in MATLAB has been designed and implemented that could record an audio message and retrieve the time-domain, frequency-domain, and statistical characteristics of the message in real time. All of the required plots, calculations, and statistics listed on the project sheet are available and functional, the bonus noise filter has been completely coded and is tunable, and the underlying code can be run on a standard MATLAB installation and is not dependent on the installation of any toolboxes that may not be available on a particular machine. Five types of tests were carried out that demonstrated correct behavior with normal, silent, and noisy tones, that the bonus noise filter is measurable and explainable on the recorded signal, and that a loaded raw data file yields the same results as the original live data file. When put together, the entire system is not only a technical choreography that is finished, but also a functioning system which demonstrates that the Fourier Transform, convolution and all other concepts of Signals and Systems actually represent the real behavior of an audio signal.

5.2 Future Improvements and Takeaways

If more time is available, the noise filter might be expanded to include an adaptive, spectral subtraction stage, so that both out-of-band and in-band overlapping noise could be treated as the same time, rather than requiring to select one type of filter (that does something) or the other (that helps with noise when present). It could also be extended to save and load previous recordings for direct use, without having to export any files, and compare them side by side directly on the GUI, to make it easier to create a small library of test recordings and compare them side by side without opening the recording tools. An additional extension that could be considered is allowing the user to adjust the cutoff frequencies of the filter (not just the sharpness) and hence making it a full general purpose band pass filter. The most valuable product of the project was, as can be easily put into words, to see for oneself how a filter's mathematical design (cutoff frequencies, filter order) manifests in an audible and measurable real-world effect.

5.3 Lessons Learned

This project was a first pass at using the Fourier Transform on a real, not an idealized signal, for hands-on experience. Backing noise, bad timing and a range that does not always neatly fit into the shape a text book derivation would imply, all of which are part of real recordings, can be a bit messier than the idealized signals used as part of coursework: learning to interpret those imperfections, rather than be confused by them, was a major part of this project. Some significant new learning also occurred in creating a GUI for MATLAB which showed up when using `uifigure` and `uigridlayout` for the first time, as well as dealing with the toolbox availability problem by implementing the FFT-based signal-processing functions manually, and building the noise filter by iteration, testing multiple design approaches (moving average, narrow bandpass, wide bandpass, and spectral subtraction) before reaching a design that produced a noise reduction that was both measurable and natural sounding. The development of routines like the PSD calculation and the bandpass filter from basic MATLAB routines, rather than using the routines provided in the toolbox, also led to a careful examination of the internal computations of these routines, as opposed to treating them like "black boxes."

5.4 Individual Work Reflection

All design, implementation, testing and documentation was done on a self-contained basis as a solo project. The staged deadlines given for the work were followed: the GUI layout and literature review was completed first, then the analysis functions were all fully implemented, then the bonus noise filter was tested and refined iteratively, and finally this report. Working alone, it meant that for any decision that had to be made from the choice of filter cutoffs, to the layout of the GUI itself, it had to be thought out and justified rationally with no second opinion to turn to, making the testing phase particularly crucial, as mistakes that could have been caught by a teammate may have slipped past. The most personal learning was the importance of testing under many deliberately different conditions, including noisy speech, clean speech and silence, as several real issues were uncovered as a by-product of that much wider testing process, both the absence of a Power Spectrum plot and an indexing error in the calculation of the spectrogram.

5.5 Impact Statement

The advantage of the system constructed in this project is that it is an educational system that allows an user to "see" the Fourier Transform of his/her voice in real time, as opposed to being an abstract concept. Having the ability to record a sentence and then see right on the screen the spectrum, the energy, and the statistics which resulted from the set of equations is a much more direct way to get his or her understanding than simply having to work through the same equations on paper. The tool could be used in the same way in the future in other classes. The system requires no additional hardware, and no extra electronic waste is produced, as it is all built on a standard laptop. It only consumes the small amount of power that's needed to run MATLAB during a recording session. A privacy concern arises as the system is designed to store the user's voice, and all recordings in this project were kept locally for use in the course, but never shared with any external services, and in future any use of the tool with other voices would require a local-only handling of the recordings. The system is not a commercial product, so it should not replace current audio-analysis applications or tasks, but its primary goal is to make it easier to comprehend how the analysis tools work inside.

A MATLAB Source Code

This appendix lists the complete MATLAB source code for the three files that make up the Sound Recognition and Visualization system.

A.1 soundRecognitionGUI.m

```
function soundRecognitionGUI
%SOUNDRECOGNITIONGUI Sound Recognition and Visualization GUI.
%   CEN320/BME320 Project - Amro Kamal Jalad (1092505)
%
%   This app lets the user:
%     - record a short audio message
%     - see the time-domain waveform
%     - see the Fourier Transform (frequency spectrum)
%     - see the Power Spectrum
%     - see the Power Spectral Density
%     - see a histogram of the signal
%     - see a spectrogram (frequency content over time)
%     - read off the Energy, Max Frequency, Mean, Std, and Variance
%     - optionally filter noise before analyzing (bonus requirement)
%     - save the recording as a labeled .wav file (raw data for
    submission)
%     - export the whole GUI as a PNG image (for the report)
%
%   This file only builds the GUI and connects buttons to actions.
    The
%   actual math is done in two separate files:
%       analyzeSignal.m - computes FFT/power
    spectrum/PSD/energy/stats/spectrogram
%       filterNoise.m   - the bonus noise filter
%
%   Requires: base MATLAB only, no toolboxes.

clc  % just clears the command window so old output doesn't
    confuse me

%% ---------------- Settings ----------------
```

```
28 Fs = 44100;            % sampling frequency (Hz) - standard CD
       quality
29 recTime = 5;           % default recording duration (seconds), user
       can change this in the GUI
30
31 %% ---------------- Shared data ----------------
32 % these need to be up here (not inside a function) so every
       callback
33 % below can see them and change them - this is how nested
       functions work
34 x  = [];   % raw recorded signal
35 xf = [];   % filtered signal (after applying the noise filter)
36 freqZoom = 5000; % default frequency-axis zoom (Hz) for
       FFT/Power/PSD plots
37                 % (5000 because that's basically where speech
                       ends, no point showing all the way to 22050)
38
39 %% ---------------- Color theme ----------------
40 % just picked colors that looked decent together, nothing fancy
41 colAccent = [0.16 0.42 0.80];  % header / play button blue
42 colRecord = [0.82 0.20 0.20];  % record button + "recording" lamp
43 colGreen  = [0.16 0.60 0.34];  % save button + "ready" lamp
44 colIdle   = [0.75 0.75 0.75];  % lamp before anything is recorded
45 colCard   = [1.00 1.00 1.00];  % panel background
46 colReset  = [0.45 0.45 0.48];  % reset button
47
48 colTime = [0.16 0.42 0.80];    % time-domain plot line
49 colFFT  = [0.47 0.24 0.66];    % FFT plot line
50 colPower = [0.13 0.55 0.45];   % power spectrum plot line
51 colPSD  = [0.85 0.55 0.10];    % PSD plot line
52 colHist = [0.82 0.32 0.24];    % histogram bars
53 % gave each plot its own color so it's easier to tell them apart
       at a glance
54
55 %% ================ Main window ================
56 % uifigure instead of the old figure() command because it plays
       nice
57 % with uigridlayout and the newer components like uilamp, uislider
       etc.
58 fig = uifigure('Name','Sound Recognition and Visualization', ...
```

```
    'Position',[50 20 1320 960],'Color',[0.95 0.96 0.97]);

% splitting the window into 2 rows: a small header on top,
    everything else below
outer = uigridlayout(fig,[2 1]);
outer.RowHeight = {64,'1x'}; % header is fixed height, rest fills
    whatever space is left
outer.Padding = [0 0 0 0];
outer.RowSpacing = 0;

% ---- Header bar ----
header =
    uipanel(outer,'BackgroundColor',colAccent,'BorderType','none');
header.Layout.Row = 1;
hg = uigridlayout(header,[2 1]);
hg.RowHeight = {'1x','1x'};
hg.Padding = [18 4 18 4];
hg.RowSpacing = 0;
hg.BackgroundColor = colAccent; % had to set this separately, the
    panel color alone doesn't show through the grid

uilabel(hg,'Text','Sound Recognition and Visualization', ...
    'FontSize',20,'FontWeight','bold','FontColor',[1 1 1]);
uilabel(hg,'Text','CEN320 / BME320 - Amro Kamal Jalad (1092505)',
    ...
    'FontSize',12,'FontColor',[0.88 0.92 1]);

% ---- Body: sidebar + plots ----
% this is the main row, split into left (controls) and right
    (plots)
body = uigridlayout(outer,[1 2]);
body.Layout.Row = 2;
body.ColumnWidth = {320,'1x'}; % sidebar fixed at 320px, plots
    take the rest
body.Padding = [14 14 14 14];
body.ColumnSpacing = 14;

%% ================ Sidebar ================
% 4 stacked panels: Recording, Noise Filter, Save/Export,
    Statistics
```

```
sidebar = uigridlayout(body,[4 1]);
sidebar.Layout.Column = 1;
sidebar.RowHeight = {270,150,155,'1x'}; % had to tweak these
    numbers a bunch until nothing overlapped
sidebar.RowSpacing = 14;
sidebar.Padding = [0 0 0 0];

% ---- Recording panel ----
recPanel =
    uipanel(sidebar,'Title','Recording','FontWeight','bold','BackgroundColor',colCard);
recPanel.Layout.Row = 1;
recGrid = uigridlayout(recPanel,[5 2]);
recGrid.RowHeight = {28,34,34,34,34,26}; % 4 buttons + duration
    field + lamp row
recGrid.ColumnWidth = {'1x','1x'};
recGrid.RowSpacing = 6;

uilabel(recGrid,'Text','Duration (s):');
dur_field =
    uieditfield(recGrid,'numeric','Value',recTime,'Limits',[1 30]);
% limited to 1-30 sec so people don't accidentally record for an
    hour

btn_record = uibutton(recGrid,'push','Text','Record', ...
    'BackgroundColor',colRecord,'FontColor',[1 1
        1],'FontWeight','bold', ...
    'ButtonPushedFcn',@record_callback);
btn_record.Layout.Column = [1 2]; % spans both columns so it's a
    full-width button

btn_play = uibutton(recGrid,'push','Text','Play', ...
    'BackgroundColor',colAccent,'FontColor',[1 1
        1],'FontWeight','bold', ...
    'ButtonPushedFcn',@play_callback);
btn_play.Layout.Column = [1 2];

btn_reset = uibutton(recGrid,'push','Text','Reset', ...
    'BackgroundColor',colReset,'FontColor',[1 1
        1],'FontWeight','bold', ...
    'ButtonPushedFcn',@reset_callback);
```

```
btn_reset.Layout.Column = [1 2];

btn_load = uibutton(recGrid,'push','Text','Load Audio File', ...
    'BackgroundColor',colAccent,'FontColor',[1 1
        1],'FontWeight','bold', ...
    'ButtonPushedFcn',@load_callback);
btn_load.Layout.Column = [1 2];
% added this one later on, doctor wanted a way to check raw data
    files
% without needing physical drag and drop (which doesn't really
    work well in base matlab)

lamp = uilamp(recGrid,'Color',colIdle); % little status light,
    changes color depending on what's happening
status_lbl = uilabel(recGrid,'Text','Idle','FontColor',[0.45 0.45
    0.45]);

% ---- Noise filter panel (bonus) ----
filterPanel = uipanel(sidebar,'Title','Noise Filter
    (Bonus)','FontWeight','bold','BackgroundColor',colCard);
filterPanel.Layout.Row = 2;
filterGrid = uigridlayout(filterPanel,[4 1]);
filterGrid.RowHeight = {26,20,32,'1x'};

chk_filter = uicheckbox(filterGrid,'Text','Apply noise
    filter','ValueChangedFcn',@filter_callback);
strength_lbl = uilabel(filterGrid,'Text','Filter sharpness: 15');
strength_slider = uislider(filterGrid,'Limits',[3 50],'Value',15,
    ...
    'MajorTicks',[3 15 30 50],'ValueChangedFcn',@strength_callback);
% slider value doesn't directly = filter order, it gets converted
    in sliderToOrder()
% lower down. kept it 3-50 because that felt like a good
    "readable" range for a slider
uilabel(filterGrid,'Text','Removes hum below 150 Hz and hiss above
    5000 Hz.', ...
    'FontColor',[0.55 0.55 0.55],'FontSize',11,'WordWrap','on');

% ---- Save / export panel ----
```

```
150 savePanel = uipanel(sidebar,'Title','Save &
        Export','FontWeight','bold','BackgroundColor',colCard);
151 savePanel.Layout.Row = 3;
152 saveGrid = uigridlayout(savePanel,[3 1]);
153 saveGrid.RowHeight = {30,34,34};
154
155 name_field = uieditfield(saveGrid,'text','Value','amro_recording');
156 btn_save = uibutton(saveGrid,'push','Text','Save as .wav', ...
157     'BackgroundColor',colGreen,'FontColor',[1 1
            1],'FontWeight','bold', ...
158     'ButtonPushedFcn',@save_callback);
159 btn_export = uibutton(saveGrid,'push','Text','Export GUI as .png',
        ...
160     'BackgroundColor',colAccent,'FontColor',[1 1
            1],'FontWeight','bold', ...
161     'ButtonPushedFcn',@export_callback);
162 % added export button so i don't need to alt+tab and use snipping
        tool every time
163 % for the report screenshots, saves a lot of time honestly
164
165 % ---- Statistics panel ----
166 statsPanel =
        uipanel(sidebar,'Title','Statistics','FontWeight','bold','BackgroundColor',colCard);
167 statsPanel.Layout.Row = 4;
168 statsGrid = uigridlayout(statsPanel,[7 1]);
169 statsGrid.RowHeight = {24,24,24,24,24,24,24};
170 statsGrid.RowSpacing = 4;
171
172 chk_fullspec = uicheckbox(statsGrid,'Text','Show full spectrum
        (0-22050 Hz)', ...
173     'FontSize',11,'ValueChangedFcn',@fullspec_callback);
174 % by default we zoom into 0-5000Hz because that's basically all
        speech,
175 % but sometimes i wanted to see the full range so added this toggle
176
177 energy_lbl = uilabel(statsGrid,'Text','Energy:
        -','FontWeight','bold');
178 maxfreq_lbl = uilabel(statsGrid,'Text','Max frequency: -');
179 mean_lbl   = uilabel(statsGrid,'Text','Mean: -');
180 std_lbl    = uilabel(statsGrid,'Text','Std deviation: -');
```

```
var_lbl    = uilabel(statsGrid,'Text','Variance: -');
psd_lbl    = uilabel(statsGrid,'Text','Peak PSD: -');
% all start as "-" until something actually gets recorded/loaded

%% ================= Plot area (3x2 grid of cards) =================
plotsGrid = uigridlayout(body,[3 2]);
plotsGrid.Layout.Column = 2;
plotsGrid.RowSpacing = 12;
plotsGrid.ColumnSpacing = 14;

% making 6 plot "cards" using my own helper function below (addPlotCard)
% so i don't have to copy paste the same uipanel/uiaxes code 6 times
ax_time = addPlotCard(1,1,'Time-Domain Signal');
ax_fft  = addPlotCard(1,2,'Fourier Transform');
ax_power = addPlotCard(2,1,'Power Spectrum');
ax_psd  = addPlotCard(2,2,'Power Spectral Density');
ax_hist = addPlotCard(3,1,'Histogram');
ax_spec = addPlotCard(3,2,'Spectrogram');

% same deal here, using styleAxes() helper so every plot looks consistent
% instead of setting grid/box/fontsize manually 6 separate times
styleAxes(ax_time, 'Time (s)','Amplitude');
styleAxes(ax_fft, 'Frequency (Hz)','Magnitude');
styleAxes(ax_power,'Frequency (Hz)','Power');
styleAxes(ax_psd, 'Frequency (Hz)','Power');
styleAxes(ax_hist, 'Amplitude','Count');
styleAxes(ax_spec, 'Time (s)','Frequency (Hz)');
ax_spec.XGrid = 'off'; % grid lines look bad over the spectrogram heatmap, turned them off just for this one
ax_spec.YGrid = 'off';

%% ================= Callback functions =================
% everything below here only runs when the user actually clicks something,
% not when the app first opens

```

```
215     function record_callback(~,~)
216         % this runs when Record is clicked. records for however many
217         % seconds the user set, then sends it off to be analyzed
218         recTime = dur_field.Value;
219         lamp.Color = colRecord;
220         status_lbl.Text = 'Recording...';
221         status_lbl.FontColor = colRecord;
222         drawnow;  % forces the GUI to actually update NOW instead
                of waiting,
223                   % otherwise the "Recording..." text wouldn't show
                      up until after recording is done
224
225         recorder = audiorecorder(Fs,16,1); % Fs, bits per sample,
                mono (1 = mono, 2 would be stereo)
226         recordblocking(recorder,recTime); % "blocking" means matlab
                just waits here until it's done, doesn't move on
227         x = getaudiodata(recorder);      % pull the actual sample
                numbers out of the recorder object
228
229         lamp.Color = colGreen;
230         status_lbl.Text = 'Recording complete';
231         status_lbl.FontColor = colGreen;
232
233         chk_filter.Value = false;        % new recording -> start
                unfiltered, don't carry over old filter state
234         xf = [];
235         updateDisplay(x);
236     end
237
238     function load_callback(~,~)
239         % lets you pick an existing audio file off your computer
                instead
240         % of recording live - this is what the doctor asked for so
                raw
241         % data files could be checked independently later
242         [file, path] = uigetfile({'*.wav;*.mp3;*.m4a;*.flac', ...
243             'Audio Files (*.wav, *.mp3, *.m4a, *.flac)'}, 'Select an
                    audio file');
244         if isequal(file,0)
```

```
245             return  % user clicked Cancel, just do nothing and stop
                    here
246         end
247
248         try
249             [loadedSig, loadedFs] = audioread(fullfile(path,file));
250             if size(loadedSig,2) > 1
251                 loadedSig = mean(loadedSig,2); % if it's stereo (2
                        columns), average them into 1 mono column
252             end
253
254             x = loadedSig;
255             Fs = loadedFs; % important - use the FILE's own sample
                    rate, not always 44100
256                            % (some files might be recorded at a
                                different rate)
257
258             lamp.Color = colGreen;
259             status_lbl.Text = ['Loaded ' file];
260             status_lbl.FontColor = colGreen;
261
262             chk_filter.Value = false;
263             xf = [];
264             updateDisplay(x);
265         catch ME
266             % if something goes wrong (corrupt file, wrong format
                    etc)
267             % don't just crash the whole app, show an error message
                    instead
268             status_lbl.Text = 'Failed to load file';
269             status_lbl.FontColor = colRecord;
270             disp(ME.message); % prints the real matlab error to the
                    command window so I can debug it
271         end
272     end
273
274     function play_callback(~,~)
275         % plays back whatever is currently on screen - if the filter
276         % checkbox is on, play the filtered version, otherwise play
                the raw one
```

```
277         if isempty(x)
278             status_lbl.Text = 'Nothing recorded yet';
279             status_lbl.FontColor = colRecord;
280             return
281         end
282         if chk_filter.Value && ~isempty(xf)
283             sound(xf,Fs);
284         else
285             sound(x,Fs);
286         end
287     end
288
289     function reset_callback(~,~)
290         % basically puts the app back to how it was when it first
              opened.
291         % useful when demoing so i don't have to close and reopen
              matlab every time
292         x = [];
293         xf = [];
294         chk_filter.Value = false;
295         lamp.Color = colIdle;
296         status_lbl.Text = 'Idle';
297         status_lbl.FontColor = [0.45 0.45 0.45];
298
299         cla(ax_time); cla(ax_fft); cla(ax_power); % cla = clear
              axes, wipes the plots but keeps the axes themselves
300         cla(ax_psd); cla(ax_hist); cla(ax_spec);
301
302         energy_lbl.Text = 'Energy: -';
303         maxfreq_lbl.Text = 'Max frequency: -';
304         mean_lbl.Text   = 'Mean: -';
305         std_lbl.Text    = 'Std deviation: -';
306         var_lbl.Text    = 'Variance: -';
307         psd_lbl.Text    = 'Peak PSD: -';
308     end
309
310     function filter_callback(~,~)
311         % runs whenever the "Apply noise filter" checkbox gets
              clicked
312         if isempty(x)
```

```
313             chk_filter.Value = false; % don't let them turn on
                    filter with no recording, nothing to filter yet
314             return
315         end
316         if chk_filter.Value
317             order = sliderToOrder(strength_slider.Value);
318             xf = filterNoise(x,Fs,order); % actual filtering happens
                    in the separate filterNoise.m file
319             updateDisplay(xf);
320         else
321             updateDisplay(x); % filter turned off, go back to
                    showing the original unfiltered signal
322         end
323     end
324
325     function strength_callback(~,~)
326         % runs every time the sharpness slider gets dragged. if the
327         % filter is already on, we re-filter right away so you can
                SEE
328         % the change live instead of having to toggle the checkbox
                again
329         v = round(strength_slider.Value);
330         strength_lbl.Text = sprintf('Filter sharpness: %d', v);
331
332         if ~isempty(x) && chk_filter.Value
333             order = sliderToOrder(strength_slider.Value);
334             xf = filterNoise(x,Fs,order);
335             updateDisplay(xf);
336         end
337     end
338
339     function order = sliderToOrder(v)
340         % converts the slider number (3-50, easy for a human to
                read)
341         % into an actual FIR filter order (odd number, roughly
                19-301 taps)
342         % higher order = sharper/more precise filter cutoff, but
                more computation
343         order = round(v)*6 + 1; % *6 then +1 just makes sure it's
                always odd, filter needs odd length
```

```
344     end
345
346     function save_callback(~,~)
347         % writes whatever is currently shown (filtered or not) out
                to an
348         % actual .wav file on disk - this is my raw data for
                submission
349         if isempty(x)
350             status_lbl.Text = 'Nothing to save yet';
351             status_lbl.FontColor = colRecord;
352             return
353         end
354         if chk_filter.Value && ~isempty(xf)
355             sigToSave = xf;
356         else
357             sigToSave = x;
358         end
359
360         fname = [name_field.Value '.wav'];
361         audiowrite(fname, sigToSave, Fs);
362         status_lbl.Text = ['Saved ' fname];
363         status_lbl.FontColor = colGreen;
364     end
365
366     function export_callback(~,~)
367         % takes a screenshot of the whole app window and saves it
                as png,
368         % saved me a ton of time when making screenshots for the
                report
369         fname = [name_field.Value '_gui.png'];
370         try
371             exportapp(fig, fname);
372             status_lbl.Text = ['Exported ' fname];
373             status_lbl.FontColor = colGreen;
374         catch ME
375             status_lbl.Text = 'Export failed - see command window';
376             status_lbl.FontColor = colRecord;
377             disp(ME.message);
378         end
379     end
```

```
380
381     function fullspec_callback(~,~)
382         % toggles between the zoomed in view (0-5000Hz, good for
383         % speech) and showing literally the whole spectrum up to
                Fs/2
384         if chk_fullspec.Value
385             freqZoom = Fs/2;
386         else
387             freqZoom = 5000;
388         end
389         % re-draw whatever's currently on screen with the new zoom
                level
390         if ~isempty(x)
391             if chk_filter.Value && ~isempty(xf)
392                 updateDisplay(xf);
393             else
394                 updateDisplay(x);
395             end
396         end
397     end
398
399 %% ================ Display helper =================
400 % this is the function that actually gets called every single time
401 % something new needs to be shown - after recording, loading,
        filtering,
402 % or even just moving the zoom checkbox
403
404     function updateDisplay(sig)
405         % takes a signal, sends it to analyzeSignal.m to get all the
406         % numbers, then dumps everything onto the plots and labels
407
408         r = analyzeSignal(sig, Fs); % all the actual math happens
                over there, not here
409
410         % Time domain
411         plot(ax_time, r.t, r.sig, 'Color',colTime,'LineWidth',1.1);
412         xlim(ax_time, [0 r.t(end)]);
413
414         % Fourier Transform
415         plot(ax_fft, r.f, r.mag, 'Color',colFFT,'LineWidth',1.1);
```

```
416         xlim(ax_fft, [0 freqZoom]);
417
418         % Power spectrum
419         plot(ax_power, r.f, r.powerSpec,
                'Color',colPower,'LineWidth',1.1);
420         xlim(ax_power, [0 freqZoom]);
421
422         % Power spectral density
423         plot(ax_psd, r.f, r.psd, 'Color',colPSD,'LineWidth',1.1);
424         xlim(ax_psd, [0 freqZoom]);
425
426         % Histogram - 50 bins looked like a good balance, not too
427         % blocky and not too noisy
428         histogram(ax_hist, r.sig, 50,
                'FaceColor',colHist,'EdgeColor','none');
429
430         % Spectrogram (frequency content over time)
431         % using dB scale (20*log10) because otherwise the loud parts
432         % totally wash out the quiet parts and you can't see
                anything
433         imagesc(ax_spec, r.specT, r.specF, 20*log10(r.specData +
                eps));
434         set(ax_spec,'YDir','normal'); % without this the
                spectrogram shows upside down for some reason
435         colormap(ax_spec,'parula');
436         ylim(ax_spec,[0 8000]); % capped at 8000 since nothing
                interesting happens above that for voice
437
438         % Statistics labels - just formatting numbers into text and
439         % sticking them onto the labels
440         energy_lbl.Text = sprintf('Energy: %.4f', r.energy);
441         maxfreq_lbl.Text = sprintf('Max frequency: %.1f Hz',
                r.maxFreq);
442         mean_lbl.Text   = sprintf('Mean: %.5f', r.meanVal);
443         std_lbl.Text    = sprintf('Std deviation: %.5f', r.stdVal);
444         var_lbl.Text    = sprintf('Variance: %.6f', r.varVal);
445         psd_lbl.Text    = sprintf('Peak PSD: %.6f', r.peakPSD);
446     end
447
448 %% ================ Layout helpers ================
```

```
% these two just exist so I'm not repeating the same lines of code
% 6 times for the plots - basically my own little shortcut
    functions

    function ax = addPlotCard(row, col, titleStr)
        % makes one "card" - a titled panel with a plot inside it -
        % and places it at the given row/col in the plots grid
        p =
            uipanel(plotsGrid,'Title',titleStr,'FontWeight','bold','BackgroundColor',colC
        p.Layout.Row = row;
        p.Layout.Column = col;
        g = uigridlayout(p,[1 1]);
        g.Padding = [6 6 6 6];
        ax = uiaxes(g);
    end

    function styleAxes(ax, xlab, ylab)
        % makes all plots look the same - same grid style, same font
        % size etc, so it doesn't look like 6 randomly different
            plots
        xlabel(ax, xlab);
        ylabel(ax, ylab);
        ax.Box = 'on';
        ax.XGrid = 'on';
        ax.YGrid = 'on';
        ax.GridAlpha = 0.15; % light grid, don't want it too
            distracting
        ax.FontSize = 10;
        ax.Toolbar.Visible = 'on'; % lets you zoom/pan on the plots
            by hovering over them
        enableDefaultInteractivity(ax);
    end

end
```

A.2 analyzeSignal.m

```
function results = analyzeSignal(sig, Fs)
```

```
%ANALYZESIGNAL Compute all required signal analysis quantities.
%
%   results = analyzeSignal(sig, Fs) takes a recorded (or filtered)
%   audio signal "sig" sampled at "Fs" Hz, and returns a struct
%   containing everything the project asks for:
%
%       results.t       - time vector (seconds)
%       results.sig     - the signal itself (column vector)
%       results.f       - frequency vector (Hz), positive
    frequencies only
%       results.mag     - Fourier Transform magnitude |X(f)|
%       results.powerSpec - power spectrum
%       results.psd     - power spectral density
%       results.maxFreq - frequency of the strongest speech
    component (Hz)
%       results.peakPSD - peak value of the PSD
%       results.energy  - signal energy
%       results.meanVal - mean of the signal
%       results.stdVal  - standard deviation of the signal
%       results.varVal  - variance of the signal
%       results.specT   - spectrogram time axis (seconds)
%       results.specF   - spectrogram frequency axis (Hz)
%       results.specData - spectrogram magnitude (freq bins x time
    frames)
%
%   This function only computes numbers - it does not plot or touch
    the
%   GUI, so it can also be called directly from the command window
    to
%   test the math on any recording, e.g.:
%       [sig,Fs] = audioread('amro_recording.wav');
%       r = analyzeSignal(sig,Fs);
%
%   kept this file completely separate from the GUI on purpose -
    makes
%   it way easier to test, i literally just ran this on a bunch of
    my
%   saved wav files from the command line to check the numbers
    before
%   ever touching the GUI code
```

```
34
35 sig = sig(:);          % make sure the signal is a column vector
36                        % (audioread sometimes gives rows depending
                              on the file, this avoids weird bugs
                              later)
37 N = length(sig);
38 t = (0:N-1)'/Fs;       % time axis, just sample number divided by
       sample rate
39
40 % ---- Fourier Transform ----
41 X = fft(sig);          % this does basically all the heavy
       lifting, everything below is just processing X
42 mag = abs(X);          % magnitude of the FFT - X itself is
       complex, we only care about size here
43 f = (0:N-1)*(Fs/N);    % frequency axis (Hz) - each fft bin
       corresponds to a specific frequency
44
45 half = 1:floor(N/2);   % keep only positive frequencies
46                        % fft gives you a mirrored/symmetric
                              result for real signals, so the
                              second
47                        % half is basically the same info
                              repeated, no point plotting it
48 f = f(half);
49 mag = mag(half);
50
51 % ---- Power spectrum ----
52 powerSpec = mag.^2 / N; % power = magnitude squared, dividing by N
       just keeps the scale sane
53
54 % ---- Max frequency component ----
55 % Ignore everything below 50 Hz: real speech starts around 85 Hz,
56 % but recordings often have low-frequency rumble/drift near 0 Hz
57 % that would otherwise dominate.
58 % (found this out the hard way - first version of this kept saying
59 % "max frequency = 8Hz" or something dumb like that because of mic
       noise)
60 minFreq = 50;
61 magSearch = mag;
```

```
magSearch(f < minFreq) = 0; % zero out anything below 50Hz so it can't win the max() below
[~, idx] = max(magSearch);
maxFreq = f(idx);

% ---- Power spectral density ----
psd = powerSpec / Fs; % dividing again by Fs so it doesn't matter how long the recording was
peakPSD = max(psd);

% ---- Energy (sum of squared samples) ----
energy = sum(sig.^2); % straight from parsevals theorem, sum of squared samples = signal energy

% ---- Statistical features ----
meanVal = mean(sig);
stdVal  = std(sig);
varVal  = var(sig);
% these 3 are just built in matlab functions, nothing fancy going on here

% ---- Spectrogram (how the frequency content changes over time) ----
% The signal is split into short overlapping frames. Each frame gets
% its own FFT, so the result is a 2D map: frequency (rows) vs time
% (columns), with color/intensity showing how strong each frequency
% is at each moment - this is what lets you literally see the
% different syllables of speech as they happen.
frameLen = 512; % tried a couple different values here, 512 gave the best balance for how long i talk
hop = 128;      % how far the window slides each step - smaller hop = smoother but slower to compute
win = hannLocal(frameLen); % windowing each little chunk before fft-ing it, same idea as the filter file

numFrames = max(1, floor((N - frameLen)/hop) + 1);
nBins = floor(frameLen/2);
specData = zeros(nBins, numFrames); % pre-allocating this so the loop below doesn't grow the array every time (slow otherwise)
```

```
92
93  for i = 0:numFrames-1
94      idx2 = i*hop + (1:frameLen);
95      idx2 = idx2(idx2 <= N);      % near the end of the signal the
            frame might run off the edge, this catches that
96      frame = zeros(frameLen,1);
97      frame(1:length(idx2)) = sig(idx2); % pad with zeros if the
            frame got cut short
98      Xs = fft(frame .* win);
99      specData(:,i+1) = abs(Xs(1:nBins)); % +1 because matlab arrays
            start at 1, not 0, easy to mess this up
100 end
101
102 specF = (0:nBins-1)' * (Fs/frameLen);
103 specT = ((0:numFrames-1)*hop + frameLen/2) / Fs; % using the
    middle of each frame as its "time", felt more accurate than the
    start
104
105 % ---- Pack everything into one struct to return ----
106 % just bundling everything up so the GUI file can grab it all in
    one go
107 % instead of me having to return like 15 separate variables
108 results.t       = t;
109 results.sig     = sig;
110 results.f       = f;
111 results.mag     = mag;
112 results.powerSpec = powerSpec;
113 results.psd     = psd;
114 results.maxFreq = maxFreq;
115 results.peakPSD = peakPSD;
116 results.energy  = energy;
117 results.meanVal = meanVal;
118 results.stdVal  = stdVal;
119 results.varVal  = varVal;
120 results.specT   = specT;
121 results.specF   = specF;
122 results.specData = specData;
123
124 end
125
```

```
126
127 function w = hannLocal(N)
128     % Manual Hann window (column vector, length N)
129     % same reasoning as filterNoise.m - avoiding the built in hann()
130     % function since it needs signal processing toolbox
131     n = (0:N-1)';
132     w = 0.5*(1 - cos(2*pi*n/(N-1)));
133 end
```

A.3 filterNoise.m

```
1  function y = filterNoise(sig, Fs, order)
2  %FILTERNOISE Bandpass noise filter tuned for a clear, reliable
       effect.
3  %
4  %   y = filterNoise(sig, Fs, order) removes low-frequency rumble/hum
5  %   (below 150 Hz) and high-frequency hiss/brightness (above 5000
       Hz),
6  %   while leaving the 150-5000 Hz range - where most of a voice's
7  %   energy and clarity lives - untouched.
8  %
9  %   These cutoffs were chosen after testing two extremes:
10 %     - 300-3400 Hz ("telephone bandwidth") noticeably cleaned the
11 %       signal, but made the voice sound thin and tinny, like an old
12 %       phone call.
13 %     - 80-8000 Hz was too close to the full voice spectrum, so it
       had
14 %       almost no audible effect.
15 %   150-5000 Hz sits between the two: a clearly audible cleanup
       effect
16 %   without sounding unnatural.
17 %
18 %   "order" controls how sharp the filter's cutoff is (how cleanly
       it
19 %   separates the passband from the rejected frequencies). Higher
       order
20 %   = sharper, more precise filtering. Always rounded up to odd.
21 %
```

```
%   Built manually as a windowed-sinc FIR bandpass filter, using only
%   conv(), sin(), and cos() - no Signal Processing Toolbox needed.
%
%   this whole file went through like 4 versions before i landed on
%   this one lol. first tried a simple moving average (too weak/muffled
%   the voice), then tried the toolbox functions (broke on machines
%   without the toolbox), so ended up just building it from scratch
%   using the sinc + window method from class

if nargin < 3
    order = 101;  % default if nobody passes an order in, just a reasonable middle value
end
if mod(order,2) == 0
    order = order + 1; % filter must be an odd length to stay symmetric
                       % (even length messes up the delay/centering math below)
end

fc1 = 150;    % lower cutoff (Hz) - anything below this is just hum/rumble, not voice
fc2 = 5000;   % upper cutoff (Hz) - anything above this is hiss, voice doesn't really go this high

% n is just a list of integers centered on 0, like -50...0...50
% this is so the filter is symmetric, which we need for it to be a real
% (not complex) filter
n = -(order-1)/2 : (order-1)/2;

% Ideal lowpass impulse responses at each cutoff, built from the
% sinc function (written manually so no toolbox is needed)
% basically: sinc function = what a "perfect" lowpass filter looks like
% in the time domain. this comes straight from the fourier transform pair
```

```
% between a rectangle (ideal frequency cutoff) and a sinc (time domain)
hLow2 = 2*fc2/Fs * sincManual(2*fc2/Fs*n);
hLow1 = 2*fc1/Fs * sincManual(2*fc1/Fs*n);

% A bandpass filter is just the difference of two lowpass filters
% (lowpass up to fc2) minus (lowpass up to fc1) = only keeps the middle band
hBand = hLow2 - hLow1;

% Apply a Hamming window to smooth the filter's response and reduce
% ringing compared to an unwindowed "ideal" filter
% without this the filter has bad ripples/ringing, learned this the hard
% way when i first tried it without a window and the output sounded weird
w = hammingManual(order)';
hBand = hBand .* w;

% Filter the signal using convolution, then trim back to the
% original signal length
% conv() makes the output longer than the input, so we have to cut off
% the extra padding on both sides to get back to the original length
yFull = conv(sig, hBand);
pad = (order-1)/2;
y = yFull(pad+1 : pad+length(sig));

end


function w = hammingManual(N)
% Manual Hamming window (column vector, length N)
% wrote this myself since hamming() is a toolbox function and i wanted
% this whole project to run on any matlab install, not just ones with
% signal processing toolbox
nn = (0:N-1)';
```

```
w = 0.54 - 0.46*cos(2*pi*nn/(N-1)); % this exact formula is just
    the standard hamming window equation
end


function y = sincManual(x)
% Manual sinc function: sin(pi*x)/(pi*x), with sinc(0) = 1
% again, avoiding the built in sinc() since that also needs a
    toolbox.
% the 0/0 case has to be handled separately because matlab will
    just
% give you NaN otherwise, even though the actual limit is 1
y = ones(size(x));
nz = x ~= 0;
y(nz) = sin(pi*x(nz)) ./ (pi*x(nz));
end
```

References

[1] A. V. Oppenheim and A. S. Willsky, *Signals and Systems*, 2nd ed. Upper Saddle River, NJ: Prentice Hall, 1996.

[2] P. D. Welch, "The use of fast Fourier transform for the estimation of power spectra: A method based on time averaging over short, modified periodograms," *IEEE Transactions on Audio and Electroacoustics*, vol. 15, no. 2, pp. 70–73, Jun. 1967.

[3] MathWorks, "Audio toolbox documentation," https://www.mathworks.com/help/audio/, Natick, MA, 2024.

[4] J. B. Allen, "Short term spectral analysis, synthesis, and modification by discrete Fourier transform," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 25, no. 3, pp. 235–238, Jun. 1977.

[5] F. J. Harris, "On the use of windows for harmonic analysis with the discrete Fourier transform," *Proceedings of the IEEE*, vol. 66, no. 1, pp. 51–83, Jan. 1978.

[6] L. R. Rabiner and R. W. Schafer, *Digital Processing of Speech Signals*. Englewood Cliffs, NJ: Prentice Hall, 1978.

[7] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed. Upper Saddle River, NJ: Prentice Hall, 2007.